TP Unidad 7 · NoSQL y nuevas formas de almacenamiento

Food Store: pedidos y detalle de pedidos en MongoDB

Integrante: Agustina Fontagnol · **Repositorio:** https://github.com/agusfonta/foodstore_bd (carpeta `entrega_tp7/`) ·
Motores: PostgreSQL 17, MongoDB 9.0.2 (instancia única local). Los datos son de prueba sintéticos, sin datos personales reales.

1. Modelo documental (consigna 1)

Subconjunto elegido: **pedidos y detalle de pedidos**. En PostgreSQL intervienen `pedidos` (PK `id`, FK `cliente_id`), `detalle_pedido` (FK `pedido_id`, `producto_id`; UNIQUE `pedido_id, producto_id`), `clientes` y `productos`. Se propone una colección `foodstore_u7.pedidos` con un documento por pedido:

```
{ _id: 3,                          // = pedidos.id
  fecha: ISODate(...), estado: "CANCELADO", total: Decimal128("4740.69"), forma_pago: "EFECTIVO",
  cliente: { id: 3, nombre, apellido, mail },          // objeto embebido
  lineas: [ { producto_id: 5, nombre: "Producto_5", cantidad: 5,
              precio_unitario: Decimal128("498.71"), subtotal: Decimal128("2493.55") }, ... ] }  //
arreglo embebido
```

Identificadores conservados: `_id` = `pedidos.id`, `cliente.id` = `clientes.id` y `lineas[].producto_id` = `productos.id`. Importes como `Decimal128` para no perder precisión.

2. Migración (consigna 2)

Se exportaron los pedidos 1 a 10 de `foodstore_desarrollo` con un SELECT de solo lectura (`jsonb_build_object`) y se cargaron con `insertMany()`. Los 10 pedidos cubren los cuatro estados y tienen entre 1 y 5 líneas; varios productos se repiten entre pedidos. No se recalculó ningún importe.

```
> mongosh --quiet entrega_tp7/migrar_pedidos.js
insertados: 10 | en coleccion: 10
> db.pedidos.aggregate([{$group:{_id:"$estado",n:{$sum:1}}},{$sort:{_id:1}}])
CANCELADO: 2 · CONFIRMADO: 3 · PENDIENTE: 2 · TERMINADO: 3
```

3. Reglas de integridad y decisión de embebido (consigna 3)

Regla conservada	Origen en PostgreSQL	Verificación en Mongo
Coherencia de importes: `subtotal = cantidad × precio_unitario` y `total` = suma de subtotales	CHECK en `detalle_pedido` y `pedidos`	0 pedidos y 0 líneas con diferencias
Cantidad > 0 y un solo renglón por producto en cada pedido	`chk_detalle_cantidad`, UNIQUE `(pedido_id, producto_id)`	0 líneas con cantidad ≤ 0; 0 pedidos con producto repetido

Dentro del documento: las líneas (no existen sin el pedido y se leen siempre juntas), el precio unitario histórico (un cambio posterior de precio no debe alterar pedidos cerrados), el nombre del producto y los datos esenciales del cliente. **En colecciones separadas:** el catálogo de productos y los clientes completos, que tienen vida propia y se reutilizan en muchos pedidos. Costo: el nombre y los datos del cliente quedan duplicados como foto histórica.

4. Consultas en MongoDB (consigna 4)

```
a) db.pedidos.find({}, {estado:1})                                  → 10 documentos (_id y estado)
b) db.pedidos.find({estado:"CONFIRMADO"}, {_id:1})                   → 3 pedidos: [1,5,9]
c) db.pedidos.find({"lineas.producto_id":6}, {_id:1})                → 3 pedidos: [2,3,4]
d) db.pedidos.find({total:{$gt:Decimal128.fromString("2134.07")}}) → 4 pedidos: [2,3,4,9]
```

Umbral de d): 2134.07, el promedio de `total` de los pedidos migrados (2134.068), para separar los pedidos de monto superior al habitual.

5. Equivalentes en PostgreSQL y verificación (consigna 5)

```
-- b)  SELECT count(*), array_agg(id ORDER BY id) FROM pedidos
--      WHERE estado='CONFIRMADO' AND NOT eliminado AND id BETWEEN 1 AND 10;      → 3, {1,5,9}
-- c)  SELECT count(*), array_agg(p.id ORDER BY p.id) FROM pedidos p
--      WHERE NOT p.eliminado AND p.id BETWEEN 1 AND 10 AND EXISTS
--        (SELECT 1 FROM detalle_pedido d WHERE d.pedido_id=p.id AND d.producto_id=6); → 3, {2,3,4}
```

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs
Estado seleccionado (`CONFIRMADO`)	3 (1, 5, 9)	3 (1, 5, 9)	Sí
Producto seleccionado (`producto_id = 6`)	3 (2, 3, 4)	3 (2, 3, 4)	Sí

Diferencia detectada: sin el filtro `id BETWEEN 1 AND 10`, PostgreSQL devuelve 50.000 pedidos `CONFIRMADO` y 12 pedidos con el producto 6, porque Mongo solo contiene el subconjunto migrado; las consultas se acotaron para comparar lo mismo. En c) se usó `EXISTS` y no `JOIN`: con el JOIN un pedido aparecería una vez por cada línea coincidente.

6. Evaluación con IA (consignas 6 y 7)

Herramienta: Claude Code (Claude Sonnet 5.5). **Prompt:** comparar PostgreSQL y MongoDB para almacenar y consultar pedidos de Food Store, indicando el modelo relacional actual (`pedidos`, `detalle_pedido` con FK, CHECK y UNIQUE), el acceso frecuente a pedidos completos y la necesidad de conservar la coherencia entre cantidades y precios. **Respuesta (resumen):** Mongo lee un pedido completo sin JOIN; sus transacciones multidocumento requieren replica set; no aplica FK ni CHECK por defecto; PostgreSQL garantiza la integridad de forma declarativa; recomendó PostgreSQL como sistema de registro.

Afirmación de la IA	Contraste en el laboratorio	Resultado
1. Mongo no aplica integridad por defecto; PostgreSQL sí.	Mongo aceptó un pedido con cliente 999999 y producto 888888 inexistentes, cantidad −5 y subtotal incoherente. PostgreSQL (`BEGIN/ROLLBACK`) rechazó los tres casos: `chk_detalle_cantidad`, `fk_pedido_cliente` y `chk_detalle_subtotal_coherente`.	Confirmada
2. Las transacciones multidocumento de Mongo requieren replica set.	En la instancia única local, la transacción falló: «This MongoDB deployment does not support retryable writes». El pedido 1 quedó intacto.	Confirmada

Referencia complementaria: https://www.mongodb.com/docs/manual/core/transactions/

7. Conclusión

Conviene mantener PostgreSQL como sistema de registro de pedidos. Primero, por la integridad: en el laboratorio PostgreSQL rechazó una cantidad negativa, un cliente inexistente y un subtotal incoherente, mientras MongoDB aceptó un documento con esos errores, porque sin validación de esquema la coherencia de cantidades y precios queda a cargo de la aplicación. Segundo, por la atomicidad: en la instalación local de MongoDB una transacción multidocumento no pudo ejecutarse y requeriría un replica set, mientras que en PostgreSQL es el comportamiento básico. A favor de MongoDB está el patrón de acceso: un pedido completo se lee con un solo `find`, sin JOIN, y el precio histórico queda congelado en el documento. Esa ventaja es real, pero no compensa perder garantías en un dominio donde los importes deben ser exactos. La opción de combinar tecnologías tiene sentido solo como copia de lectura derivada, con PostgreSQL como única fuente de escritura y validación, y únicamente si el volumen de lecturas de pedidos completos lo justifica. Para el tamaño actual del proyecto, esa complejidad adicional no se justifica.

Archivos adjuntos

`consultas_mongodb.js`, `consultas_postgresql.sql`, `migrar_pedidos.js`, `sql/exportar_pedidos_json.sql`, `verificacion_integridad.js` y `docs/duia_tp7.md` (Declaración de Uso de IA).